

Hacettepe University

Computer Engineering Department

BBM204 Software Practicum II - 2025 Spring

Programming Assignment 1

March 21, 2025

Student name:
Öykü Tuğyan

Student Number:
b2220765029

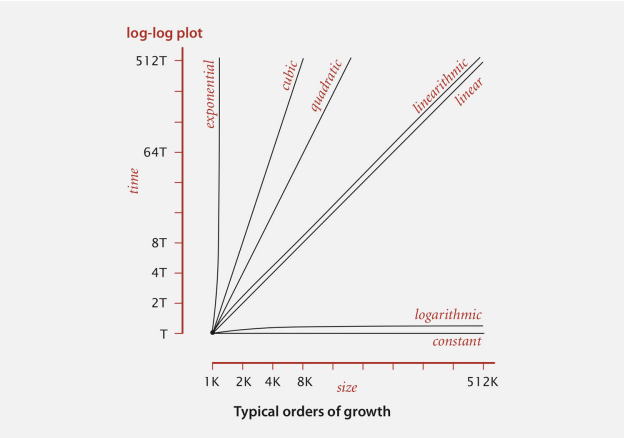


Figure 1: Complexity Graph[1]

1 Problem Definition

Analysis of Algorithms is a fundamental aspect of computer science that involves evaluating performance of algorithms and programs. Efficiency is measured in terms of time and space [2]. It is mainly used to predict performance.

A "Sorting Algorithm" is used to rearrange a given array or list of elements in order [3]. Sorting plays a crucial role in the management of data sets. In this project, our goal is to apply different algorithms to different partitions of the data set and demonstrate the relationship between the runtime of each algorithm and their theoretical asymptotic complexities.

We will evaluate five different sorting algorithms:
1. Comb Sort
2. Insertion Sort
3. Shaker Sort
4. Shell sort
5. Radix Sort

In addition, we will test these algorithms on different types of subsets:
1. Randomly arranged subset
2. Sorted subset
3. Reversely-sorted subset

With these experiments, we can understand how each algorithm performs with different data sizes and structures and validate their theoretical execution time.

2 Solution Implementations

2.1 Sorting Algorithm 1: Comb Sort

```java
public class CombSort {
    /**
     * Sorts the given array using combSort
     * @param arrayA    Array to be sorted
     */
    public static void combSort(int[] arrayA) {
        int gap = arrayA.length;
        double shrink = 1.3;
        boolean sorted = false;
        while (!sorted) {
            gap = max(1, (int) Math.floor(gap/shrink));
            sorted = (gap == 1);
            for (int i = 0; i < arrayA.length - gap; i++) {
                if (arrayA[i] > arrayA[i + gap]) {
                    swap(arrayA, i, i + gap);
                    sorted = false;
                }
            }
        }
    }
    /**
     * Finds the max
     * @param a     The first integer
     * @param b     The second integer
     * @return      Returns the max integer
     */
    public static int max(int a, int b) {
        if (a > b) {
            return a;
        } else {return b;}
    }
    /**
     * Swaps the values
     * @param array     Array to make switch operation
     * @param a     Index of the first element
     * @param b     Index of the second element
     */
    public static void swap(int[] array, int a, int b) {
        int temp = array[a];
        array[a] = array[b];
        array[b] = temp;
    }
}
```

2.2 Sorting Algorithm 2: Insertion Sort

```
44 public class InsertionSort {
45     /**
46      * Sorts the given array using insertionSort
47      * @param arrayA     Array to be sorted
48      */
49     public static void insertionSort(int[] arrayA) {
50         for (int j = 1; j < arrayA.length; j++) {
51             int key = arrayA[j];
52             int i = j - 1;
53             while (i >= 0 && arrayA[i] > key) {
54                 arrayA[i + 1] = arrayA[i];
55                 i = i - 1;
56             }
57             arrayA[i + 1] = key;
58         }
59     }
60 }
```

2.3 Sorting Algorithm 3: Shaker Sort

```
61 public class ShakerSort {
62     /**
63      * Sorts the given array using shakerSort
64      * @param arrayA    Array to be sorted
65      */
66     public static void shakerSort(int[] arrayA) {
67         boolean swapped = true;
68         while (swapped) {
69             swapped = false;
70             for (int i = 0; i <= arrayA.length - 2; i++) {
71                 if (arrayA[i] > arrayA[i + 1]) {
72                     swap(arrayA, i, i + 1);
73                     swapped = true;
74                 }
75             }
76             if (!swapped) {
77                 break;
78             }
79             swapped = false;
80             for (int i = arrayA.length - 2; i>= 0 ; i--) {
81                 if (arrayA[i] > arrayA[i + 1]) {
82                     swap(arrayA, i, i + 1);
83                     swapped = true;
84                 }
```

```
85                }
86            }
87        }
88        /**
89         * Swaps the values
90         * @param array     Array to make switch operation
91         * @param a     Index of the first element
92         * @param b     Index of the second element
93         */
94        public static void swap(int[] array, int a, int b) {
95            int temp = array[a];
96            array[a] = array[b];
97            array[b] = temp;
98        }
99    }
```

2.4 Sorting Algorithm 4: Shell Sort

```
100 public class ShellSort {
101     /**
102      * Sorts the given array using shellSort
103      * @param arrayA    Array to be sorted
104      */
105     public static void shellSort(int[] arrayA) {
106         int n = arrayA.length;
107         int gap = n / 2;
108         while (gap > 0) {
109             for (int i = gap; i < n; i++) {
110                 int temp = arrayA[i];
111                 int j = i;
112                 while (j >= gap && arrayA[j - gap] > temp) {
113                     arrayA[j] = arrayA[j - gap];
114                     j = j - gap;
115                 }
116                 arrayA[j] = temp;
117             }
118             gap = gap / 2;
119         }
120     }
121 }
```

2.5 Sorting Algorithm 5: Radix Sort

```
122 public class RadixSort {
123     /**
124      * Sorts the given array using radixSort
125      * @param arrayA Array to be sorted
126      * @param d Number of digits
127      * @return The sorted array
128      */
129     public static int[] radixSort(int[] arrayA, int d) {
130         for (int pos = 0; pos < d; pos++) {
131             arrayA = countingSort(arrayA, pos);
132         }
133         return arrayA;
134     }
135     /**
136      * Helper function
137      * @param arrayA Array to be sorted
138      * @param pos Digit position
139      * @return The output array */
140     public static int[] countingSort(int[] arrayA, int pos) {
141         int[] count = new int[10];
142         int size = arrayA.length;
143         int[] output = new int[size];
144         for (int i = 0; i < size; i++) {
145             int digit = getDigit(arrayA[i], pos);
146             count[digit]++;
147         }
148         for (int j = 1; j < 10; j++) {count[j] += count[j - 1];}
149         for (int k = size - 1; k >= 0; k--) {
150             int digit = getDigit(arrayA[k], pos);
151             output[count[digit] - 1] = arrayA[k];
152             count[digit]--;
153         }
154         System.arraycopy(output, 0, arrayA, 0, size);
155         return arrayA;
156     }
157     /**
158      * Returns the digit at a specific position
159      * @param number The number The position of the digit
160      * @param position
161      * @return The wanted digit at the given position*/
162     public static int getDigit(int number, int position) {
163         if (position <= 0) {return 0;}
164         return (number / (int) Math.pow(10, position - 1)) % 10;
165     }
166 }
```

3 Results, Analysis, Discussion

Running time test results for sorting algorithms are given in Table 1:

Table 1: Results of the running time tests performed for varying input sizes (in ms).

Algorithm	**Input Size n**									
	500	**1000**	**2000**	**4000**	**8000**	**16000**	**32000**	**64000**	**128000**	**250000**
	Random Input Data Timing Results in ms									
Comb sort	4.8594	0.1033	0.2233	0.4322	0.8565	1.7807	4.0177	8.4047	17.9028	43.3151
Insertion sort	1.773	0.3344	0.9957	3.7258	13.166	55.7314	177.0872	734.5093	2986.5058	13140.8845
Shaker sort	2.67	1.039	1.5689	5.4952	18.9357	88.9089	448.4546	2392.4665	11939.365	50610.5789
Shell sort	1.5686	0.4369	0.2674	0.5591	1.0999	2.4654	4.4306	9.9088	21.7364	43.3644
Radix sort	9.6981	0.826	1.2504	1.7393	2.8919	5.7133	9.9141	20.8652	50.72	82.6909
	Sorted Input Data Timing Results in ms									
Comb sort	1.4698	0.0519	0.113	0.2358	0.4443	1.0078	2.3582	4.8898	10.8607	22.79
Insertion sort	0.0935	0.1315	0.0157	0.0306	0.0503	0.1132	0.1795	0.4101	0.6942	1.348
Shaker sort	0.1422	0.0228	0.0109	0.0106	0.0181	0.0355	0.0955	0.1812	0.3553	0.7215
Shell sort	0.829	0.3654	0.1348	0.328	0.5557	1.2269	2.2589	4.926	10.55	21.7582
Radix sort	5.0841	0.6231	0.9666	1.699	2.7234	5.6823	10.3095	22.0971	54.2194	83.9685
	Reversely Sorted Input Data Timing Results in ms									
Comb sort	0.3662	0.0616	0.1292	0.2605	0.5137	1.1196	2.4953	5.9107	11.5058	23.8419
Insertion sort	0.4388	0.5949	1.8483	6.8357	25.2756	95.0454	341.9039	1485.756	5779.4013	20720.6088
Shaker sort	0.6766	0.9806	3.0472	11.1667	45.0229	151.9072	627.9337	2675.1591	10566.1663	37014.6783
Shell sort	0.1118	0.0693	0.1513	0.2866	0.6202	1.194	2.4658	5.4273	14.2207	22.7344
Radix sort	5.2227	2.1525	0.9076	1.4733	2.8632	5.8008	10.0735	23.1195	54.2715	83.6606

Complexity analysis results are given in Table 2 and Table 3:

Table 2: Computational complexity comparison of the given algorithms.

Algorithm	**Best Case**	**Average Case**	**Worst Case**
Comb sort	$\Omega(nlogn)$	$\Theta(n^2)$	$O(n^2)$
Insertion sort	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$
Shaker sort	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$
Shell sort	$\Omega(nlogn)$	$\Theta(n^2)$	$O(n^2)$
Radix sort	$\Omega(nk)$	$\Theta(nk)$	$O(nk)$

n :input size, k: range of the input

Table 3: Auxiliary space complexity of the given algorithms.

Algorithm	**Auxiliary Space Complexity**
Comb sort	$O(1)$
Insertion sort	$O(1)$
Shaker sort	$O(1)$
Shell sort	$O(1)$
Radix sort	$O(n+k)$

Output

Experiments on the Given Random Data (Size: 500)

- Mean: 4.8594 milliseconds for Comb Sort
- Mean: 1.773 milliseconds for Insertion Sort
- Mean: 2.67 milliseconds for Shaker Sort
- Mean: 1.5686 milliseconds for Shell Sort
- Mean: 9.6981 milliseconds for Radix Sort

Experiments on the Sorted Data (Size: 500)

- Mean: 1.4698 milliseconds for Comb Sort
- Mean: 0.0935 milliseconds for Insertion Sort
- Mean: 0.1422 milliseconds for Shaker Sort
- Mean: 0.829 milliseconds for Shell Sort
- Mean: 5.0841 milliseconds for Radix Sort

Experiments on the Reversely Sorted Data (Size: 500)

- Mean: 0.3662 milliseconds for Comb Sort
- Mean: 0.4388 milliseconds for Insertion Sort
- Mean: 0.6766 milliseconds for Shaker Sort
- Mean: 0.1118 milliseconds for Shell Sort
- Mean: 5.2227 milliseconds for Radix Sort

Experiments on the Given Random Data (Size: 1000)

- Mean: 0.1033 milliseconds for Comb Sort
- Mean: 0.3344 milliseconds for Insertion Sort
- Mean: 1.039 milliseconds for Shaker Sort
- Mean: 0.4369 milliseconds for Shell Sort
- Mean: 0.826 milliseconds for Radix Sort

Experiments on the Sorted Data (Size: 1000)

- Mean: 0.0519 milliseconds for Comb Sort
- Mean: 0.1315 milliseconds for Insertion Sort
- Mean: 0.0228 milliseconds for Shaker Sort
- Mean: 0.3654 milliseconds for Shell Sort
- Mean: 0.6231 milliseconds for Radix Sort

Experiments on the Reversely Sorted Data (Size: 1000)

- Mean: 0.0616 milliseconds for Comb Sort
- Mean: 0.5949 milliseconds for Insertion Sort
- Mean: 0.9806 milliseconds for Shaker Sort
- Mean: 0.0693 milliseconds for Shell Sort
- Mean: 2.1525 milliseconds for Radix Sort

Experiments on the Given Random Data (Size: 2000)

- Mean: 0.2233 milliseconds for Comb Sort
- Mean: 0.9957 milliseconds for Insertion Sort
- Mean: 1.5689 milliseconds for Shaker Sort
- Mean: 0.2674 milliseconds for Shell Sort
- Mean: 1.2504 milliseconds for Radix Sort

Experiments on the Sorted Data (Size: 2000)

- Mean: 0.113 milliseconds for Comb Sort
- Mean: 0.0157 milliseconds for Insertion Sort
- Mean: 0.0109 milliseconds for Shaker Sort
- Mean: 0.1348 milliseconds for Shell Sort
- Mean: 0.9666 milliseconds for Radix Sort

Experiments on the Reversely Sorted Data (Size: 2000)

- Mean: 0.1292 milliseconds for Comb Sort
- Mean: 1.8483 milliseconds for Insertion Sort
- Mean: 3.0472 milliseconds for Shaker Sort
- Mean: 0.1513 milliseconds for Shell Sort
- Mean: 0.9076 milliseconds for Radix Sort

Experiments on the Given Random Data (Size: 4000)

- Mean: 0.4322 milliseconds for Comb Sort
- Mean: 3.7258 milliseconds for Insertion Sort
- Mean: 5.4952 milliseconds for Shaker Sort
- Mean: 0.5591 milliseconds for Shell Sort
- Mean: 1.7393 milliseconds for Radix Sort

Experiments on the Sorted Data (Size: 4000)

- Mean: 0.2358 milliseconds for Comb Sort
- Mean: 0.0306 milliseconds for Insertion Sort
- Mean: 0.0106 milliseconds for Shaker Sort
- Mean: 0.328 milliseconds for Shell Sort
- Mean: 1.699 milliseconds for Radix Sort

Experiments on the Reversely Sorted Data (Size: 4000)

- Mean: 0.2605 milliseconds for Comb Sort
- Mean: 6.8357 milliseconds for Insertion Sort
- Mean: 11.1667 milliseconds for Shaker Sort
- Mean: 0.2866 milliseconds for Shell Sort
- Mean: 1.4733 milliseconds for Radix Sort

Experiments on the Given Random Data (Size: 8000)

- Mean: 0.8565 milliseconds for Comb Sort
- Mean: 13.166 milliseconds for Insertion Sort
- Mean: 18.9357 milliseconds for Shaker Sort
- Mean: 1.0999 milliseconds for Shell Sort
- Mean: 2.8919 milliseconds for Radix Sort

Experiments on the Sorted Data (Size: 8000)

- Mean: 0.4443 milliseconds for Comb Sort
- Mean: 0.0503 milliseconds for Insertion Sort
- Mean: 0.0181 milliseconds for Shaker Sort
- Mean: 0.5557 milliseconds for Shell Sort
- Mean: 2.7234 milliseconds for Radix Sort

Experiments on the Reversely Sorted Data (Size: 8000)

- Mean: 0.5137 milliseconds for Comb Sort
- Mean: 25.2756 milliseconds for Insertion Sort
- Mean: 45.0229 milliseconds for Shaker Sort
- Mean: 0.6202 milliseconds for Shell Sort
- Mean: 2.8632 milliseconds for Radix Sort

Experiments on the Given Random Data (Size: 16000)

- Mean: 1.7807 milliseconds for Comb Sort
- Mean: 55.7314 milliseconds for Insertion Sort
- Mean: 88.9089 milliseconds for Shaker Sort
- Mean: 2.4654 milliseconds for Shell Sort
- Mean: 5.7133 milliseconds for Radix Sort

Experiments on the Sorted Data (Size: 16000)

- Mean: 1.0078 milliseconds for Comb Sort
- Mean: 0.1132 milliseconds for Insertion Sort
- Mean: 0.0355 milliseconds for Shaker Sort
- Mean: 1.2269 milliseconds for Shell Sort
- Mean: 5.6823 milliseconds for Radix Sort

Experiments on the Reversely Sorted Data (Size: 16000)

- Mean: 1.1196 milliseconds for Comb Sort
- Mean: 95.0454 milliseconds for Insertion Sort
- Mean: 151.9072 milliseconds for Shaker Sort
- Mean: 1.194 milliseconds for Shell Sort
- Mean: 5.8008 milliseconds for Radix Sort

Experiments on the Given Random Data (Size: 32000)

- Mean: 4.0177 milliseconds for Comb Sort
- Mean: 177.0872 milliseconds for Insertion Sort
- Mean: 448.4546 milliseconds for Shaker Sort
- Mean: 4.4306 milliseconds for Shell Sort
- Mean: 9.9141 milliseconds for Radix Sort

Experiments on the Sorted Data (Size: 32000)

- Mean: 2.3582 milliseconds for Comb Sort
- Mean: 0.1795 milliseconds for Insertion Sort
- Mean: 0.0955 milliseconds for Shaker Sort
- Mean: 2.2589 milliseconds for Shell Sort
- Mean: 10.3095 milliseconds for Radix Sort

Experiments on the Reversely Sorted Data (Size: 32000)

- Mean: 2.4953 milliseconds for Comb Sort
- Mean: 341.9039 milliseconds for Insertion Sort
- Mean: 627.9337 milliseconds for Shaker Sort
- Mean: 2.4658 milliseconds for Shell Sort
- Mean: 10.0735 milliseconds for Radix Sort

Experiments on the Given Random Data (Size: 64000)

- Mean: 8.4047 milliseconds for Comb Sort
- Mean: 734.5093 milliseconds for Insertion Sort
- Mean: 2392.4665 milliseconds for Shaker Sort
- Mean: 9.9088 milliseconds for Shell Sort
- Mean: 20.8652 milliseconds for Radix Sort

Experiments on the Sorted Data (Size: 64000)

- Mean: 4.8898 milliseconds for Comb Sort
- Mean: 0.4101 milliseconds for Insertion Sort
- Mean: 0.1812 milliseconds for Shaker Sort
- Mean: 4.926 milliseconds for Shell Sort
- Mean: 22.0971 milliseconds for Radix Sort

Experiments on the Reversely Sorted Data (Size: 64000)

- Mean: 5.9107 milliseconds for Comb Sort
- Mean: 1485.756 milliseconds for Insertion Sort
- Mean: 2675.1591 milliseconds for Shaker Sort
- Mean: 5.4273 milliseconds for Shell Sort
- Mean: 23.1195 milliseconds for Radix Sort

Experiments on the Given Random Data (Size: 128000)

- Mean: 17.9028 milliseconds for Comb Sort
- Mean: 2986.5058 milliseconds for Insertion Sort
- Mean: 11939.365 milliseconds for Shaker Sort
- Mean: 21.7364 milliseconds for Shell Sort
- Mean: 50.72 milliseconds for Radix Sort

Experiments on the Sorted Data (Size: 128000)

- Mean: 10.8607 milliseconds for Comb Sort
- Mean: 0.6942 milliseconds for Insertion Sort
- Mean: 0.3553 milliseconds for Shaker Sort
- Mean: 10.55 milliseconds for Shell Sort
- Mean: 54.2194 milliseconds for Radix Sort

Experiments on the Reversely Sorted Data (Size: 128000)

- Mean: 11.5058 milliseconds for Comb Sort
- Mean: 5779.4013 milliseconds for Insertion Sort
- Mean: 10566.1663 milliseconds for Shaker Sort
- Mean: 14.2207 milliseconds for Shell Sort
- Mean: 54.2715 milliseconds for Radix Sort

Experiments on the Given Random Data (Size: 250000)

- Mean: 43.3151 milliseconds for Comb Sort
- Mean: 13140.8845 milliseconds for Insertion Sort
- Mean: 50610.5789 milliseconds for Shaker Sort
- Mean: 43.3644 milliseconds for Shell Sort
- Mean: 82.6909 milliseconds for Radix Sort

Experiments on the Sorted Data (Size: 250000)

- Mean: 22.79 milliseconds for Comb Sort
- Mean: 1.348 milliseconds for Insertion Sort
- Mean: 0.7215 milliseconds for Shaker Sort
- Mean: 21.7582 milliseconds for Shell Sort
- Mean: 83.9685 milliseconds for Radix Sort

Experiments on the Reversely Sorted Data (Size: 250000)

- Mean: 23.8419 milliseconds for Comb Sort
- Mean: 20720.6088 milliseconds for Insertion Sort
- Mean: 37014.6783 milliseconds for Shaker Sort
- Mean: 22.7344 milliseconds for Shell Sort
- Mean: 83.6606 milliseconds for Radix Sort

Analysis

What are the best, average, and worst cases for the given algorithms in terms of the given input data to be sorted?
1. For Comb Sort, random data is the worst. Sorted and reversely sorted data are close to each other, but sorted data is a bit better.
2. For Insertion Sort, reversely sorted data is the worst, requires maximum swaps, and sorted data is definitely the best, it is nearly linear.
3. For Shaker Sort, reversely sorted data is the worst, requires maximum swaps, and sorted data is definitely the best, it is nearly linear. Its similar to Insertion Sort.
4. For Shell Sort, random data is the worst, sorted, and reversely sorted data are close to each other, its hard to say one is better than the other.
5. For Radix Sort, all the data types are similar to each other. Since it is a non-comparison-based algorithm, it has a linear complexity regardless of input order.

Do the obtained results (running times of your algorithm implementations) match their theoretical asymptotic complexities?
The experimental results align well with their theoretical complexities 2 . Here is the explanation of each algorithm:
1. Comb Sort:
It starts nearly linear, but as the input size grows, it increases quadratically.
2. Insertion Sort:
For the already sorted input data, this algorithm has the best performance - it is linear. However, for larger input data, there is an obvious increase and it makes the complexity n^2. In addition, random and reversely sorted inputs worsen the performance.
3. Shaker Sort:
The results are similar to insertion sort, the best case is linear, and there is a quadratic increase for larger input data. In addition, random and reversely sorted inputs worsen the performance as before.
4. Shell Sort:
Unlike Insertion Sort and Shaker Sort, Shell Sort performs better on larger input sizes due to reduced swaps from gap-based sorting. The results confirm sub-quadratic behavior. Even in the worst case, it remains more efficient than simple quadratic sorting algorithms.
5. Radix Sort:
It is a non-comparison-based algorithm, so it has a nearly linear performance. For each input type, every size has similar results. It handles all input types efficiently.

Do Comb Sort and Shaker Sort improve performance compared to Bubble Sort? If so, how do their approaches contribute to this improvement?
Yes, they both can improve performance compared to Bubble Sort.
1. Comb Sort:
Since it uses a gap between elements instead of just comparing the adjacent ones, it helps to move

elements further and increases efficiency. This gap at the start allows faster sorting. Even though it has the worst case n^2 like Bubble Sort, it's faster.
2. Shaker Sort:
It look at both directions (left-to-right and right-to-left) instead of only moving from left to right. Even though it has the worst case n^2 like Bubble Sort, it's faster too.

Is Shell Sort perform better than Insertion Sort for larger datasets? Under what conditions does Insertion Sort still perform well?
Yes, Shell sort generally performs better for larger datasets. As the input size grows, Shell Sort becomes more efficient due to its gap-based approach. When we look at the table 1, we can see an obvious difference between the timings of Shell Sort and Insertion Sort as input size increases. For both random data and reversely sorted data, Shell Sort performs better. However, Insertion Sort shows better performance when the data is already sorted.

Given that Radix Sort is a non-comparison-based sorting algorithm, how does it handle large numerical ranges efficiently?
Radix Sort is an algorithm that uses digits for comparison. It doesn't compare the numbers, so it's a non-comparison-based algorithm. The time complexity of Radix Sort is determined by the number of digits. Even if the range of the numbers is large, this algorithm doesn't compare these numbers directly. Therefore, Radix Sort can handle large numerical ranges more efficiently than compared to comparison-based algorithms.

References

- [1]: https://treeindev.net/img/articles/algorithm-complexity-analysis/algorithm-complexity-analysis1.jpg
- [2]: "Analysis of Algorithms", 15 Mar, 2025,https://www.geeksforgeeks.org/analysis-of-algorithms/
- [3]: "Sorting Algorithms", 10 Jan, 2025, https://www.geeksforgeeks.org/sorting-algorithms/

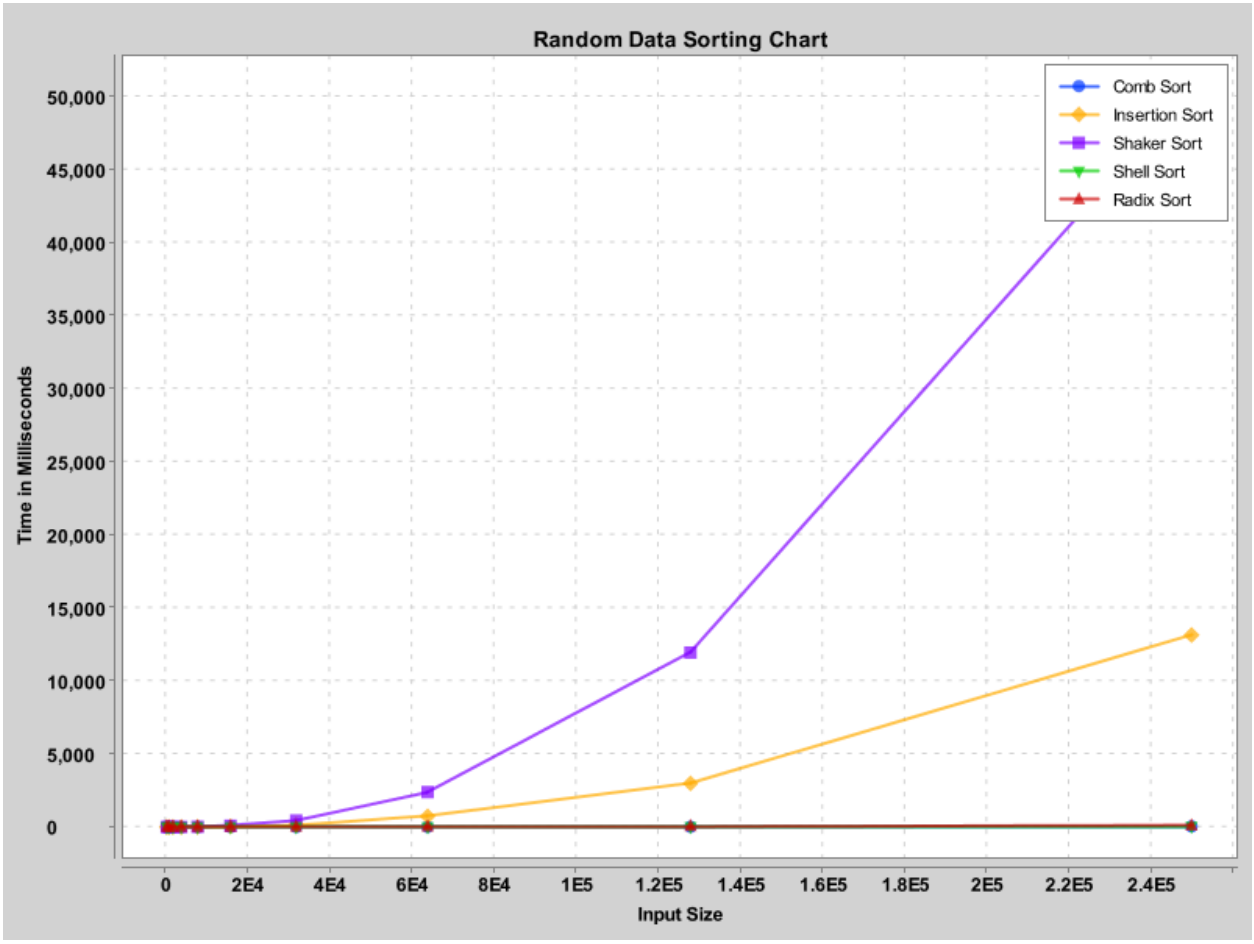


Figure 2: Random Data Sorting Chart

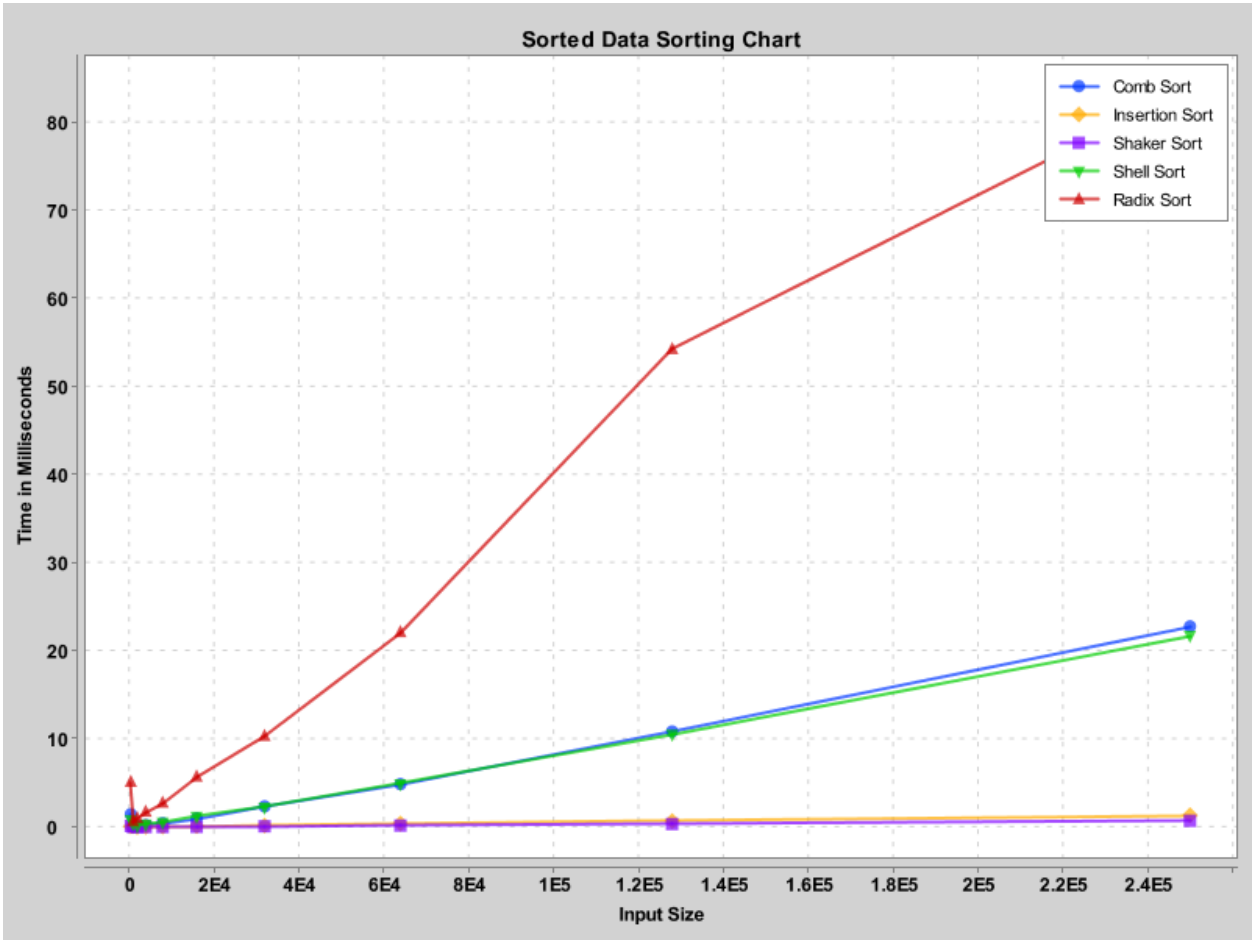


Figure 3: Sorted Data Sorting Chart

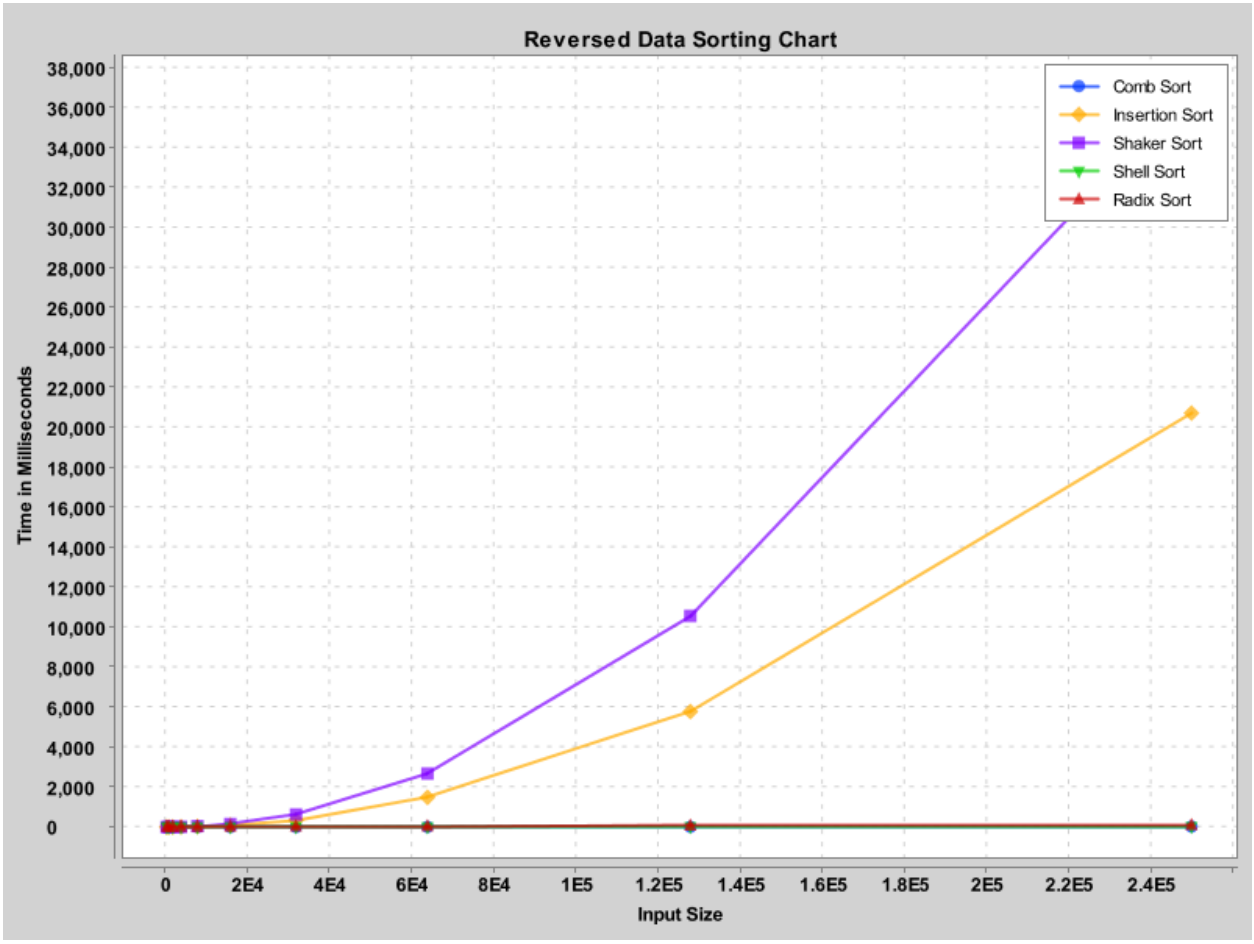


Figure 4: Reversed Data Sorting Chart

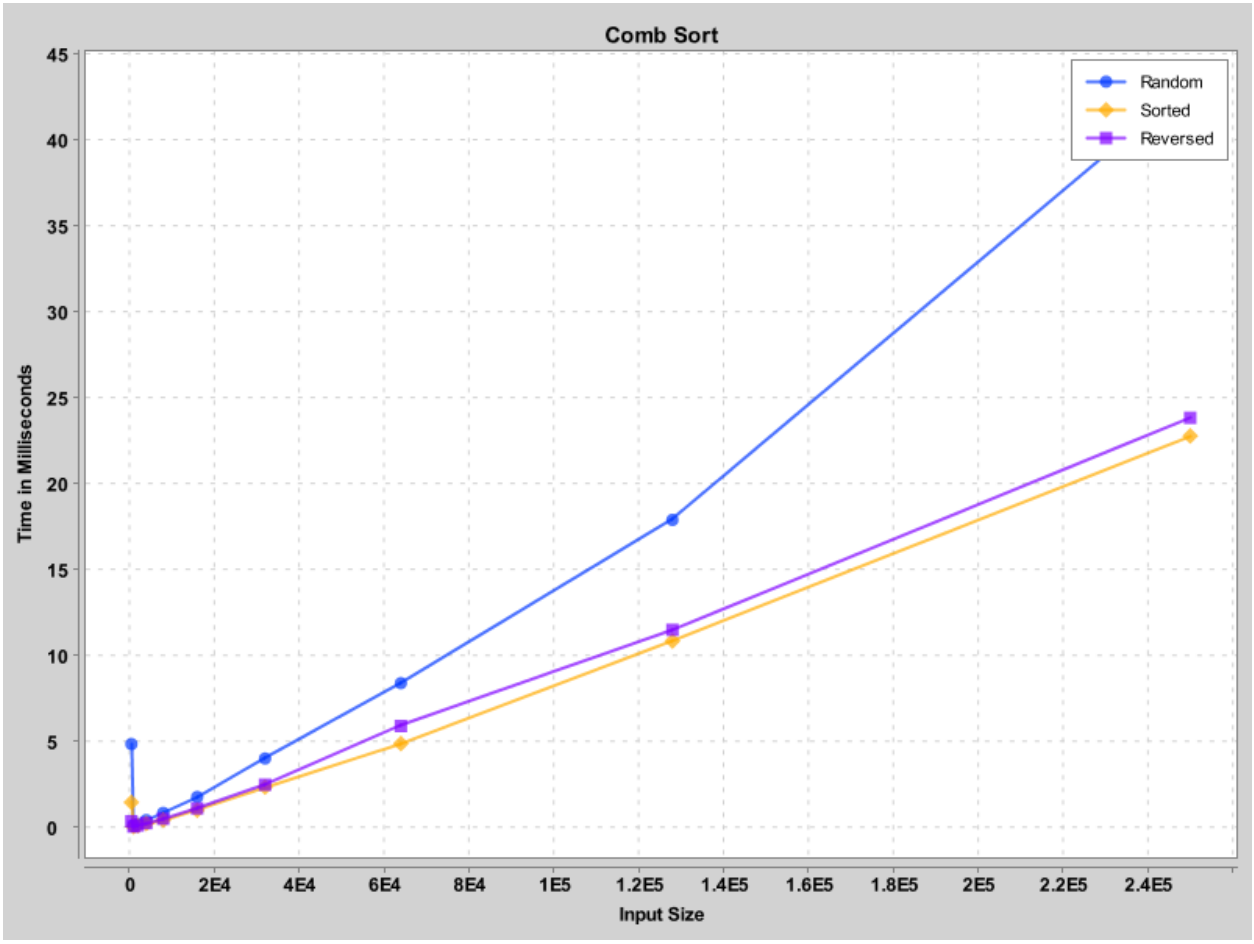


Figure 5: Comb Sort

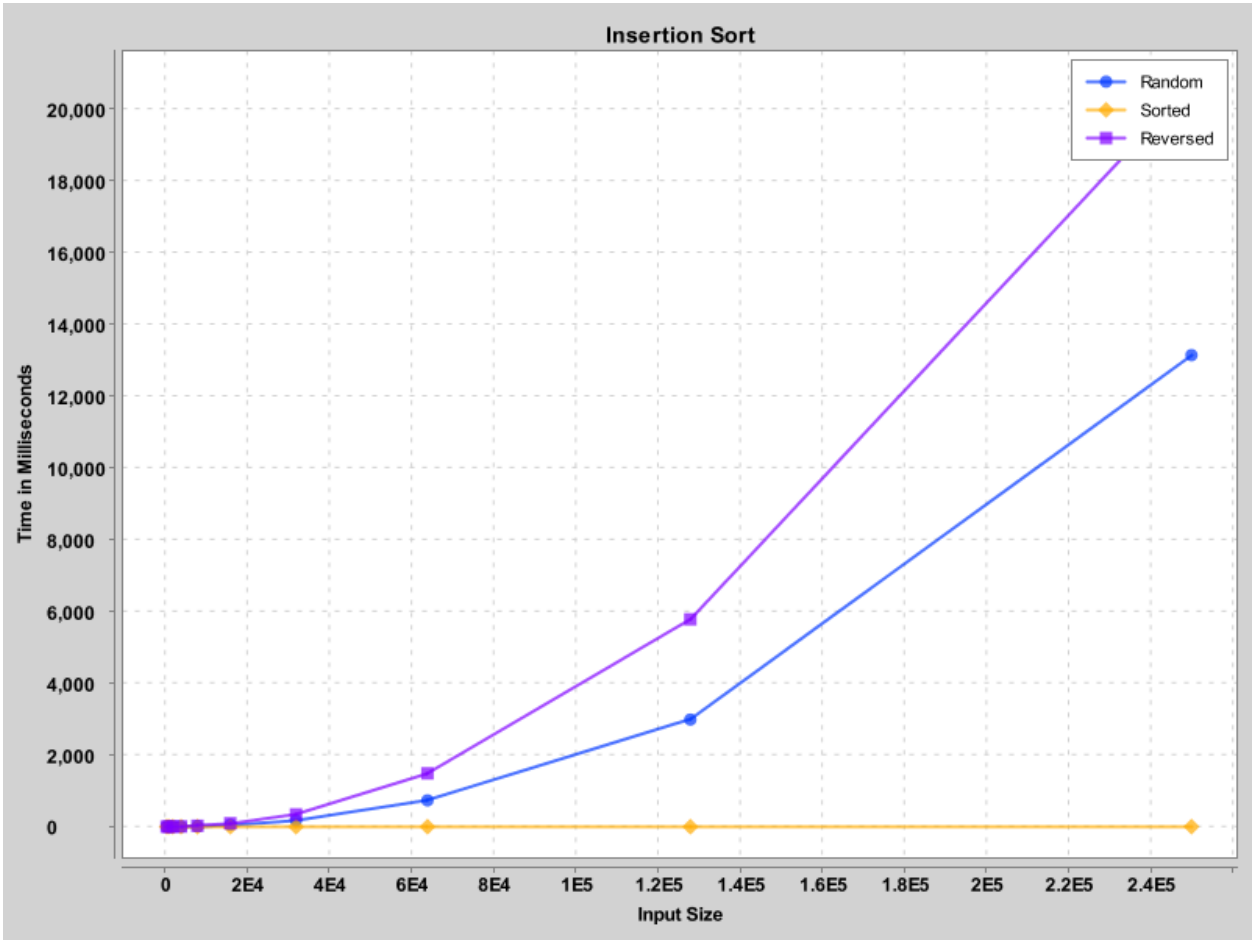


Figure 6: Insertion Sort

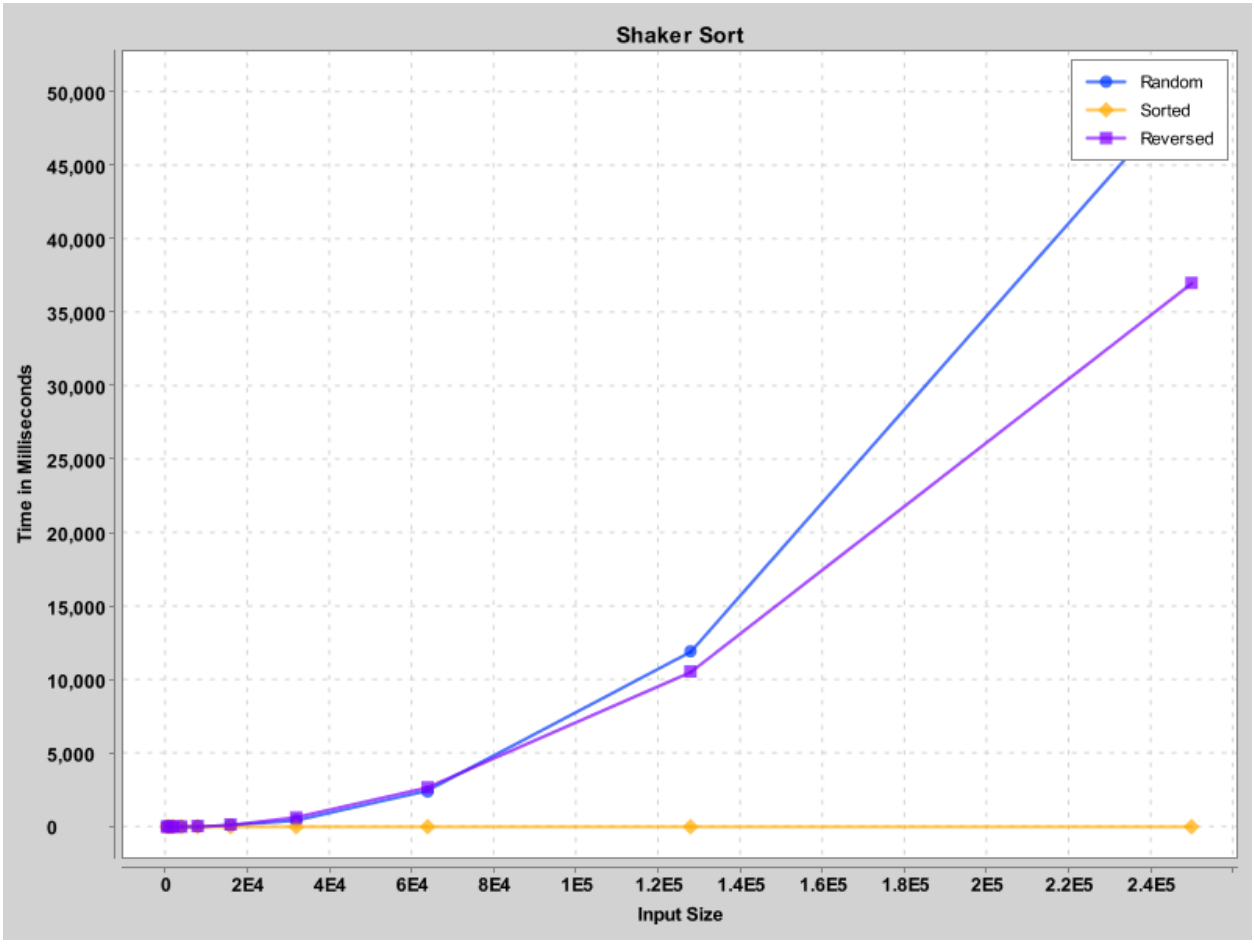


Figure 7: Shaker Sort

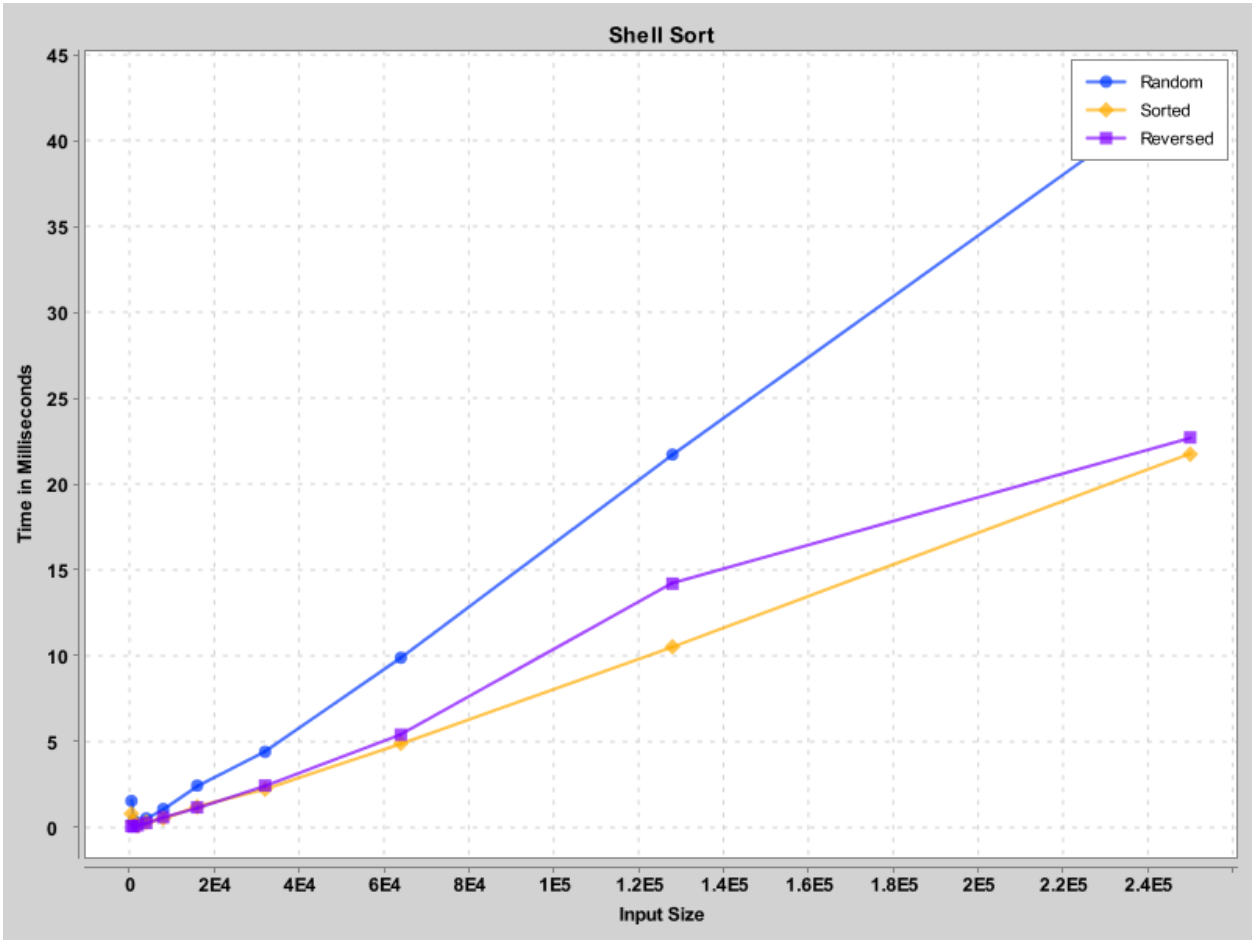


Figure 8: Shell Sort

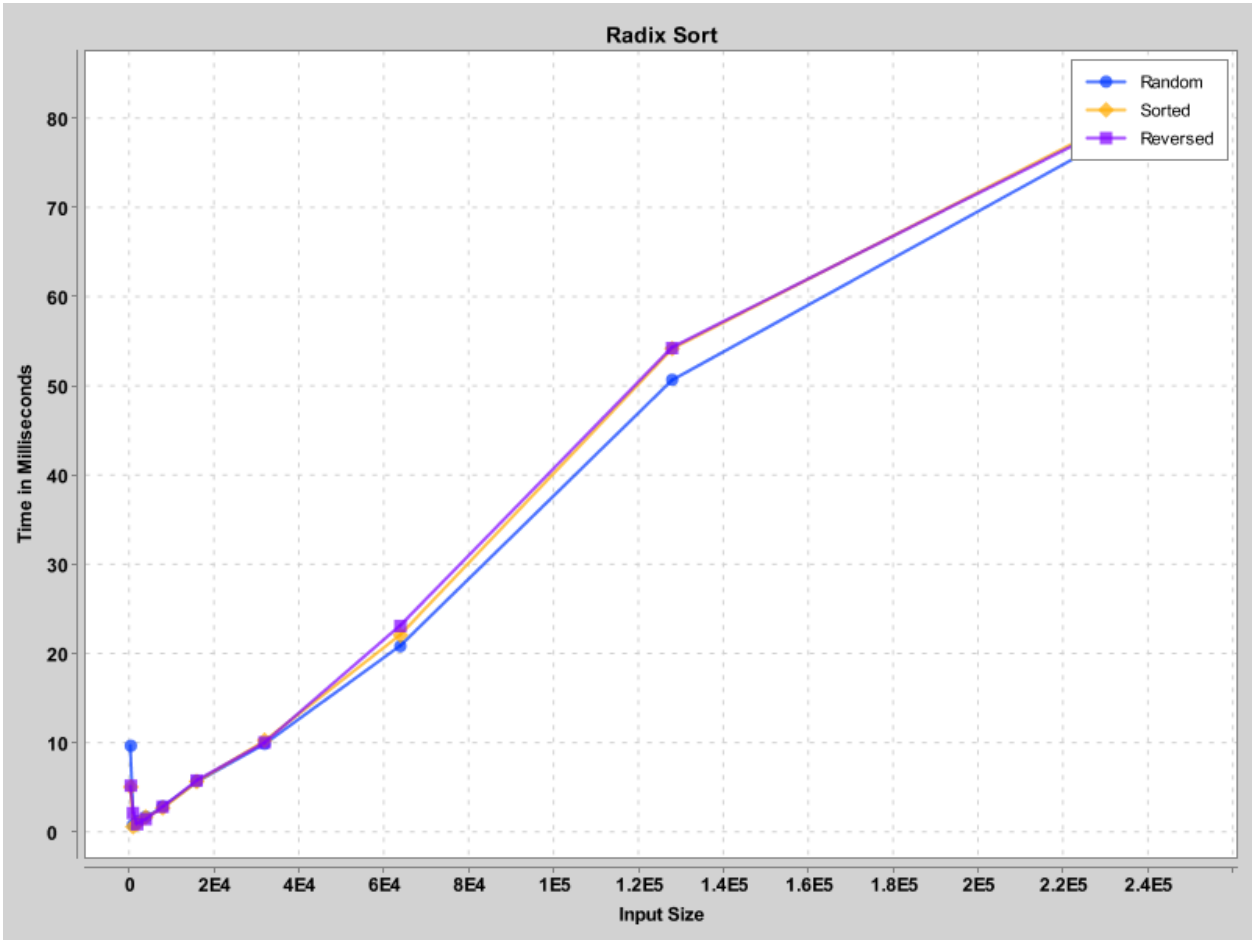


Figure 9: Radix Sort